

LLM 预训练数据层

Data Pipeline

从数据来源、解析、过滤、去重、质量评分到混合与采样的系统工程

Know-Why × Know-How × 实验驱动的数据配方

研究版本：2026-08

基于公开论文与一手技术报告整理

执行摘要：数据层真正优化的对象是什么

核心判断：预训练数据工程不是“清洗文本”的 ETL，而是在有限计算预算下主动设计训练分布。模型最终学习的是被采样后的 $P_{train}(x)$ ，因此来源、过滤、去重、质量阈值与采样权重都会直接改变能力结构。

现代公开研究已经把数据管线从“经验规则集合”推进为“可实验、可评估、可优化”的系统：FineWeb 用 70+ 个小模型做数据 ablation；DCLM 通过 416 组实验比较抽取、去重和模型过滤；DoReMi 与 RegMix 进一步把数据混合权重本身视为优化变量。[1][2][6][7]

最重要的工程矛盾是 Quality - Quantity - Diversity 三角。激进模型过滤可以在短训练 horizon 下显著提高单位 token 价值，但可能删除 90% 左右的数据；Nemotron-CC 指出，当目标是 10T-15T 级长 horizon 训练时，“高质量 token 的比例”之外，还必须关注“独特真实 token 的绝对数量”。[2][13]

核心变量	真正含义	错误直觉	更好的工程问题
Quality	每个 token 的有效学习信号密度	“过滤越狠越好”	在目标训练 horizon 下，保留多少质量层级最优？
Quantity	可提供的独特 token 数	“token 越多越好”	新增 token 是新信息，还是重复/低价值噪声？
Diversity	主题、语言、文体、任务结构覆盖	“高质量=百科/教材风格”	是否因过滤而丢失尾部主题、口语、文化与长尾语言？
Mixture	训练时各源被看到的概率	“按原始体量自然采样”	哪个 domain 的边际收益最大？是否需要上采样？
Horizon	模型总共训练多少 token	常被忽略	短 horizon 和长 horizon 应使用相同质量阈值吗？

1. 数据层全景：从 Raw Internet 到 Training Batch

Source Acquisition: Web / Books / Academic / Code / Q&A / Math / Synthetic / Multilingual
Raw Archive: WARC / HTML / PDF / Git Repo / JSON / Dumps
Parsing & Extraction: 正文、结构、代码、公式、元数据
Normalization: Unicode / Language ID / Encoding / Canonicalization
Deduplication: URL → Document → Paragraph → Exact/Fuzzy/Semantic
Quality & Safety Filtering: Rules → PPL → Classifier → LLM Judge
PII / License / Governance / Benchmark Decontamination
Quality Bucketing & Metadata Warehouse
Data Mixture / Sampling / Curriculum
Tokenizer → Packing → Training Batch

Know-why：如果只优化某一个步骤，容易发生“局部最优”：例如全局去重看似减少冗余，却可能改变各 crawl 的质量分布；FineWeb 的实验中，全局 MinHash 去重反而保留了更差的旧 crawl 数据，因此最终采用逐 snapshot 独立去重。[1]

2. 数据来源体系：不同来源究竟贡献什么能力

数据类型	典型内容	主要能力贡献	优势	主要问题	处理重点	典型公开实践
Web	网站、博客、论坛、新闻、文档	通用知识、自然语言、多样文体、长尾主题	规模最大、更新快、覆盖广	广告/模板/垃圾文本/重复/AI 生成内容/版权与隐私复杂	正文抽取、URL 过滤、语言识别、质量评分、去重	C4、RefinedWeb、FineWeb、DCLM、Nemotron-CC
Wikipedia / Reference	百科、Wikibooks、参考条目	事实密度、实体关系、规范表达	结构清晰、可验证、跨主题	文体单一，不能代表自然互联网	版本、语言、模板与引用清理；合理上采样	Dolma、Pile、ROOTS
Books	小说、教材、专业书、公共领域书籍	长上下文、叙事、知识深度、稳定文体	长文连贯、编辑质量高	许可复杂、OCR/版式、重复版本	版权/许可、章节结构、OCR 质量、版本去重	Dolma Gutenberg、Pile Books
Academic	论文、技术报告、学术 PDF	STEM、术语、论证结构、引用关系	知识密度高、逻辑结构强	PDF 抽取困难、公式/表格损失、领域偏置	布局恢复、公式/引用保真、metadata 对齐	arXiv/peS2o 类、Dolma scientific
Code	Git repo、源码、README、issue 文档	程序语义、结构化推理、工具与 API 知识	语法严格、可执行结构丰富	vendored code、生成文件、重复 fork、许可证/PII	许可、repo 级去重、文件类型过滤、secret/PII 扫描	The Stack、StarCoderData
Q&A / Forum	StackExchange、Reddit、技术问答	问题→解释/解决方案、口语、经验知识	天然具备任务/解释结构	事实性和礼貌性不稳定、重复/引用多	线程结构、投票/质量信号、引用与签名清理	Pile StackExchange、Dolma StackExchange
Math	数学网页、教材、证明、题解、LaTeX	计算、证明、符号推理、形式化结构	单位 token 推理密度高	HTML→text 容易破坏公式，规模相对小	MathJax/KaTeX/MathML 恢复、公式规范化、领域过滤	OpenWebMath、Nemotron-CC-Math
Synthetic	教材、练习、重写、推理轨迹	定向补齐稀缺技能、提高结构化质量	可控、可扩展、能覆盖难例	幻觉、风格单一、分布自循环、model collapse	Grounding、验证、去重、多生成器、真实数据保留	phi-1、Nemotron-CC rephrasing
Multilingual	多语种 Web、百科、新闻、文化内容	跨语言知识、翻译迁移、区域与文化覆盖	扩大模型服务范围	语言识别错误、低资源语言极不平衡、过滤器英语偏置	语言自适应阈值、去重、质量/重复联合平衡	CulturaX、ROOTS、FineWeb2

一个关键反直觉是：所谓“高质量来源”并不自动意味着“最佳数据混合”。RegMix 的实验显示，Web corpus 与下游性能之间可能存在比 Wikipedia 更强的正相关，而且各 domain 之间存在复杂交互，因此混合权重最好通过实验而不是常识决定。[7]

3. Web：为什么它仍然是大模型预训练的主矿脉

Web 的最大价值不是“网页多”，而是它把书籍、教程、百科、论坛、产品文档、代码解释、学术传播、个人经验等大量异构分布统一暴露在一个可扩展采集面上。FineWeb 从 96 个 Common Crawl snapshot 构建 15T token；RefinedWeb 说明经过合适过滤和去重，纯 Web 数据也能形成强基座；DCLM 则提供 240T token 级 Common Crawl 标准池用于数据实验。[1][2][4]

Web 层级	典型噪声	为什么危险	推荐处理
URL/Domain	成人站、SEO farm、镜像、垃圾域名	在解析前就浪费大量 CPU/存储	domain/URL blocklist + allow/deny metadata
HTML 模板	导航、cookie、页脚、广告、相关文章	高频模板被模型过度学习	从 WARC 重新抽取正文；DOM/boilerplate removal
Document	关键词堆砌、乱码、低信息密度列表	降低单位 token 学习价值	长度/标点/字符分布/重复行等启发式
Cross-document	转载、镜像、聚合、站群	扭曲概率分布、提高记忆与污染	MinHash / exact substring / Bloom filter
Semantic	同义改写、模板换词	表面不同但信息冗余	embedding/semantic dedup，仅在有益时使用

Web 层级	典型噪声	为什么危险	推荐处理
Generated content	机器翻译、LLM 批量文章	可能放大错误和收缩尾部分布	来源标记、AI-content signals、真实数据保留、抽样审计

4. 解析与正文抽取：第一个真正影响模型能力的环节

“WARC 里有网页”不等于“模型看到了网页正文”。FineWeb 直接比较 Common Crawl 的 WET 文本与从 WARC 使用 trafilatura 重新抽取的正文，发现 WET 保留了过多菜单和 boilerplate，而自定义抽取产生的训练模型明显更强。[1] 这说明 extraction 不是格式转换，而是第一层质量模型。

原始格式	主要挑战	应保留	应删除/隔离	工程建议
WARC/HTML	模板、脚本、DOM 碎片、动态渲染	正文、标题、列表、代码块、结构信号	菜单、cookie、广告、脚本、重复页脚	先保留 raw archive，再产 canonical text，支持重跑
PDF	多栏、页眉页脚、公式、表格、扫描页	标题层级、段落顺序、公式/表格、引用	页码、水印、重复页眉	布局感知抽取；科学 PDF 与普通 PDF 分开管线
Git Repo	vendored、minified、binary、生成文件	源码、README、测试、文档、依赖上下文	node_modules、build artifact、锁文件视目标处理	repo-aware parsing；保留 repo/file metadata
Forum/Q&A	引用嵌套、签名、投票、评论	问答关系、accepted answer、质量信号	导航、用户签名、重复引用	保留 thread structure 而非扁平化
Math HTML	MathJax/KaTeX/MathML、图片公式	公式、变量、上下文、证明结构	渲染噪声	先识别数学节点，再统一转 LaTeX

实践原则：Raw data 与 normalized data 必须分层保存。解析器未来会升级，如果只保留“第一次抽出来的纯文本”，就失去了重建数据集的能力。

5. 质量过滤：Rule、Perplexity、Classifier、LLM Judge 怎么组合

方法	信号	优点	典型风险	适合位置
规则/Heuristic	长度、标点、字符比例、重复行、停用词	快、便宜、可解释	阈值跨语言/领域失效；容易误删长尾	最前置粗筛 + 明显垃圾
Perplexity	参考 LM 对文档的惊讶程度	连续评分、实现简单	把“像参考语料”误当“质量”；领域偏置	通用质量特征之一，不宜单独决定
轻量分类器	fastText / linear classifier	高吞吐、可学习“高质量”概念	正样本定义决定偏见；容易学 spurious features	Web 主过滤器 / 质量分桶
Embedding 分类	语义向量 + 线性模型	能捕获语义层质量	成本高于 fastText、特征可能不稳定	中等规模二级筛选
LLM Judge	教育性、结构、可读性、知识密度	语义判断灵活	昂贵、Judge 偏见、不可完全复现	标注教师、少量高价值分桶/合成校验

DCLM 的大规模对比特别有价值：在 1B-1x 规模比较 PageRank、SemDedup、BGE 特征分类、AskLLM、perplexity、top-k logits 与 fastText，最终 fastText 质量分类器表现最好；进一步的 ablation 表明，用 OpenHermes 2.5 + ELI5 构造正样本并保留 top-10% 的策略优于多种传统参考语料。[2]

Know-why：“质量分类器”真正学习的不是客观质量，而是“像正样本的程度”。因此正样本的选择就是隐式能力规格书：用 Wikipedia 做正样本，会偏向百科风格；用高质量解释型数据做正样本，更可能偏向清晰解释和任务可用性。

6. 启发式过滤：不要复制阈值，要复制“产生阈值的方法”

FineWeb 的方法论比具体阈值更值得复制：先在相对高/低质量集合上统计 50+ 文档特征，寻找分布差异，再选候选阈值，最后通过 28B token 级小模型训练验证。最终保留的三类规则分别针对“句末标点比例低、重复行字符比例高、短行比例高”等现象，联合删除约 22% token，并带来 aggregate benchmark 提升。[1]

错误做法	为什么错	更可靠做法
直接照搬 C4/Gopher/FineWeb 阈值	语言、网页年代、解析器和目标模型不同	重新计算统计分布 + 小模型 ablation
几十条规则一次性叠加	无法知道哪个规则真正有效，也无法发现规则互相抵消	单因素/小组合实验，再逐步 stack
只看过滤比例	“删除得多”不是质量指标	比较固定 token 预算下的模型性能
只人工看样本	人类判断与模型有效学习信号不完全一致	人工审计 + 下游 ablation 双轨
阈值全语言共享	字符、标点、词长统计跨语言差异巨大	language-specific / adaptive thresholds

7. 去重：从“删重复”升级为“控制信息冗余与采样偏差”

层级	Key/算法	优势	主要风险	典型用途
URL exact	canonical URL / hash	极便宜、前置收益大	不同 URL 可能同文；同 URL 内容可变	同 snapshot / archive 快速裁剪
Document exact	content hash / Bloom	准确、吞吐高	无法识别轻微改写	镜像/完全复制
Paragraph exact	段落 hash / Bloom	清模板/署名/公共段落	可能破坏上下文	后置 boilerplate 清理
Fuzzy document	MinHash + LSH / n-gram	适合大规模近重复	阈值和粒度影响大	转载、轻微编辑、镜像
Exact substring	suffix array / n-gram matching	识别长重复片段	工程资源较重	跨文档长段落重复
Semantic	embedding clustering	识别同义内容	可能错误删除多样表达；成本高	知识冗余控制/研究性流程

Lee 等人的经典研究发现，现有语言模型数据存在大量近重复，去重可显著降低逐字记忆并改善训练效率，还可降低 train-test overlap。[5] 但 FineWeb 给出了更细的警告：跨 96 个 crawl 做全局 MinHash 去重时，旧 crawl 最多被删掉约 90%，留下的 10% 反而更差；逐 snapshot 去重表现更好。[1]

Dolma 采用三阶段思路：URL exact → document exact → paragraph-level dedup，并强调前两步应尽早做以降低后续处理成本，而 paragraph 去重放在质量/content filtering 之后，以减少破坏内容分析的风险。[3]

可迁移结论：Dedup 是一个 sampling policy，而不是纯清洗动作。你删掉哪个副本，就在改变时间、域名、语言、来源和质量的边际分布。因此“保留哪个副本”必须有策略。

8. Wikipedia / Books / Academic / Q&A：为什么“参考型数据”仍值得上采样

来源	稀缺价值	不应过度依赖的原因	建议处理
Wikipedia	实体关系密集、结构稳定、多语言链接	风格高度百科化，缺少真实交互与长尾文体	作为 reference domain；适度上采样；去模板/引用噪声
Books	长程连贯、编辑质量、叙事和专业知识	版权/版本/OCR 问题；时效性较低	合法许可优先；章节结构保留；版本级去重
Academic	高密度 STEM、定义-论证-证据结构	PDF 解析损失；引用语言与普通用户分布差异	公式/表格/引用保真；领域标签和年份 metadata
Q&A	问题-解释-反馈结构；贴近 instruction behavior	事实错误、口语噪声、社区偏见	利用 accepted/upvote 等质量信号；保留线程关系

Dolma 的混合实验说明 source distribution 会直接关联下游能力：排除 code 会提高 code 数据上的 perplexity；上采样论文、Wikipedia 等 reference material 会降低学术域 perplexity。因此“高质量来源”的意义更多是补充 Web 不稳定覆盖的能力维度，而不是简单替代 Web。[3]

9. Code 数据：必须从“文件数据”升级到“仓库数据”

The Stack 以 3.1TB、30 种编程语言的 permissively licensed source code 为目标，并报告 near-dedup 对代码模型性能有显著提升，同时提供 opt-out 与数据治理机制。[11] StarCoderBase 后续在 The Stack 派生数据上训练 1T token，并加入 PII redaction 和 attribution tracing。[11]

Code 管线问题	普通文本做法为什么不够	工程化处理
Fork / mirror	文件 hash 会错过 repo 级复制关系	repo-level lineage + fuzzy file dedup
Vendor / dependency	“合法源码”但信息价值重复巨大	识别 vendor、third_party、package cache
Generated / minified	语法合法但学习价值低	文件路径 + entropy/line length + generator header
Secret / PII	源码中可能含邮箱、token、key、个人信息	secret scanner + PII detector + allow/deny policy
License	文件和 repo license 可能不同	保存 license provenance；按训练/发布策略过滤
Context	单文件缺失 import / API / tests 上下文	保留 repo、path、neighbor metadata，必要时 repo packing

10. Math 数据：最难的不是筛数学，而是“不要把数学抽坏”

OpenWebMath 的关键贡献之一是指出传统 Web 文本预处理经常破坏数学符号，因此专门恢复网页中的文本与 LaTeX，并做 boilerplate removal、quality filtering 和 dedup；其 14.7B token 数据在小规模实验中显示高质量数学 Web token 具有很高的单位 token 价值。[12]

到 2025 年，Nemotron-CC-Math 进一步采用面向科学内容的抽取，处理 MathJax、KaTeX、MathML，并用 layout-aware rendering 与 LLM cleaning 修复公式和结构，构建 133B token 级高质量数学语料。[14]

数学数据失败模式	结果	防线
公式被删成空白	模型只看到“设…因此…”但关键关系消失	MathML/MathJax/KaTeX 节点抽取并转统一 LaTeX
上下标/分式顺序破坏	符号语义错误	保留结构树，不只取 rendered text
证明与导航混在一起	推理轨迹被模板污染	domain-specific boilerplate removal
题目/答案重复转载	benchmark contamination 与记忆升高	数学专用 near-dedup + eval decontamination

11. Synthetic Data：它是“数据变换器”，不是无限知识源

合成方式	目标	优势	主要风险	更安全/有效的用法
Textbook generation	把知识重写成教学结构	清晰、层次化、可控难度	生成器知识错误、风格收缩	grounded source + 事实验证 + 多模板
Exercise generation	生成题目/代码练习	可覆盖稀缺技能与难度梯度	答案错误、模板泄漏	执行器/证明器/单元测试验证
Rephrasing	降低原始 Web 噪声	保留主题同时改善格式	改写引入事实偏差	保留 source linkage；只做 transformation

合成方式	目标	优势	主要风险	更安全/有效的用法
Synthetic reasoning	构造推理轨迹	强化复杂任务结构	伪推理、模式单一	verifier / outcome filtering

phi-1 展示了“textbook quality Web + synthetic textbooks/exercises”的高数据效率路线。[15] Nemotron-CC 的 rephrasing 路线更谨慎：把模型当成文本变换器，而不是知识库，用 Wikipedia 风格重写低质量 Web 文档；其 ablation 观察到平均分提升，但也指出某些任务出现下降，可能来自合成时引入 misinformation。[13]

更长期的风险来自递归合成。Nature 2024 的 model collapse 研究说明，当后代生成模型不断以先前模型生成的数据替代真实分布时，低概率尾部会逐渐丢失并积累误差。因此工程上应把 synthetic data 当作“受控增益层”，保留足量真实/人类数据作为锚点。[16]

12. Multilingual：不能把英语过滤器平移到 1000 种语言

CulturaX 通过语言识别、URL filtering、metric cleaning、document refinement 与 dedup 建立 167 语言、6.3T token 级语料。[9] FineWeb2 在 2025 年进一步强调：不同语言必须自适应过滤与去重，并提出同时考虑 duplication count 与 quality 的重平衡方法，最终扩展到 1000+ 语言。[8]

问题	英语中心方案的失败方式	多语种解决方向
Language ID	短文本/代码混合/相近语言误判	document+segment 多层语言信号；置信度分桶
标点/词长规则	不同脚本无空格或标点体系不同	语言级统计阈值，不共享绝对数值
Perplexity 参考	Wikipedia 覆盖和风格差异巨大	每语言/语言族参考模型或自适应 percentile
Quality classifier	英语“好文章”概念不等于其他文化语境	本地高质量正样本 + 语言自适应模型
Sampling	高资源语言自然吞噬 token budget	temperature/UniMax 类重平衡 + 任务目标约束
Dedup	低资源语言重复率高，激进去重会把数据删空	quality × duplication 联合权重，而不是二值删除

13. PII、版权、许可与治理：数据工程必须保留“可追溯性”

公开数据集越来越把治理能力纳入数据管线：FineWeb 公布在 release 中匿名化 email 与 public IP；The Stack 建立 permissive license subset、搜索与 opt-out 机制；ROOTS/BLOOM 则强调数据治理与可检索透明度。[1][11][20]

治理维度	最低工程能力	为什么必须在管线里做
Provenance	source URL / crawl / document id / timestamp	支持追踪、删除、复现和质量分析
License	domain/repo/file license metadata	决定是否允许训练、发布与再分发
PII	email/IP/phone/address/identity signals	降低隐私泄漏与记忆风险
Opt-out / deletion	stable document identity + lineage	没有 lineage 就无法可靠“删除某条数据”
Safety labels	adult/toxicity/hate 等标签而非只二值删除	支持不同模型/地区的可配置 policy
Dataset card	来源、版本、过滤率、限制、已知偏差	让训练结果可解释、可复现、可审计

重要提醒：Blocklist 很容易产生分布副作用。对 C4 的系统分析发现，某些 blocklist 过滤会不成比例地删除涉及少数群体的文本。因此安全过滤应记录被删分布并做 bias audit，而不是只统计“删了多少”。[19]

14. Benchmark Decontamination：不是为了“干净好看”，而是为了让 Eval 仍有意义

训练集包含 benchmark 输入、答案或大量近似版本，会使评测分数混入记忆效应。Lee 等人已展示 dedup 可以减少 train-test overlap；后续专门研究进一步表明 text contamination 与 ground-truth contamination 都会影响模型表现，简单 n-gram 定义也存在局限。[5][25]

层级	检测对象	典型技术	注意点
Exact	题目/答案完全出现	hash / exact substring	易实现但漏掉改写
N-gram	局部连续片段重叠	n-gram overlap	阈值依 benchmark 长度调节
Fuzzy	轻微编辑/格式变化	MinHash / edit similarity	可能误删同主题正常文本
Semantic	同义改写/翻译版本	embedding similarity	成本高、定义争议大
Temporal	benchmark 发布后数据	crawl timestamp cutoff	需要可靠时间 metadata

15. 从“Keep/Drop”转向 Quality Bucketing

二值过滤的问题是把质量分数转成不可逆删除。更现代的做法是先保留多档质量 bucket，再在训练 mixture 阶段决定每档看到多少次。Nemotron-CC 明确将多个质量分类器 ensemble 后划成 5 个质量等级，目标是同时保留高质量信号与足够 unique tokens。[13]

Bucket	可能内容	短 horizon 采样	长 horizon 采样	工程角色
Q5 极高	教材、精炼解释、高质量参考	高	中高	快速建立核心能力
Q4 高	清晰文章/技术文档	高	高	主训练骨架
Q3 中	普通可读 Web	中	高	扩大知识和多样性
Q2 低	格式噪声但含有效信息	低/0	低~中；可重写	补长尾与 long-horizon unique tokens
Q1 垃圾/风险	spam、乱码、恶意模板	0	0	删除/隔离

16. Data Mixture：真正的训练数据不是“数据集”，而是 Sampling Distribution

设 domain i 的可用 token 数为 D_i ，训练权重为 w_i 。模型实际看到的数据分布不是磁盘上的原始比例，而是采样过程定义的 $P_{train}(domain=i)=w_i$ 。上采样某 domain 等价于给它更高的梯度预算。

策略	做法	优点	风险/限制
Natural sampling	按原始 token 体量采样	简单、接近数据现实	Web/英语吞噬其他 domain
Manual mixture	专家指定比例	可快速定向	依赖直觉，难发现 domain interaction
Temperature / balancing	压缩高资源、抬升低资源	多语言常用	超参依赖
DoReMi	小 proxy + Group DRO 学 domain weights	无需下游任务即可优化混合	需定义 domain、proxy 与目标
RegMix	训练大量小模型 + 回归预测 mixture→performance	显式建模 domain interaction；可搜索	前期需多次小模型训练
Late-stage curriculum	后期切换/增强高质量或专门数据	单位 token 收益高	需要把握切换时机与遗忘

DoReMi 使用 280M proxy 学到 domain weights，再用于 8B 模型，报告在 The Pile 上以 2.6× 更少训练 steps 达到 baseline accuracy。[6] RegMix 则训练 512 个 1M 模型探索 mixture，再用回归选择大模型配方，并发现不同域交互常违背人工直觉。[7]

17. Short-Horizon vs Long-Horizon：数据配方必须绑定训练长度

问题	短 horizon（例如几十 B~数百 B token）	长 horizon（数 T~15T token）
目标	最大化单位 token 信息密度	最大化总学习量 + 控制重复
质量阈值	可以更激进	不能只保留 top 10%
Unique tokens	重要但不绝对	成为核心约束
中等质量 Web	常被降权/删除	可作为多样性与长尾知识池
Synthetic	少量高质量补强	必须控制比例、重复和分布漂移
Curriculum	相对简单	更适合分阶段 quality buckets / annealing

这也是 FineWeb-Edu/DCLM 与 Nemotron-CC 看似相反、实则互补的地方：前者证明 aggressive model filtering 可以极大提升短期单位 token 质量；后者指出若目标训练长度高达 15T token，只留下 10% 数据会很快进入重复或数据不足状态。[1][2][13]

18. 数据工程的核心实验法：固定模型与 token，只改变数据

真正能把 Data Pipeline 做好的团队，不是拥有最多规则，而是拥有“数据 ablation 工厂”。FineWeb 在固定 1.71B 模型、固定 token 预算和评测集的情况下比较 extraction / dedup / filters；DCLM 则建立可在 412M~7B 规模上比较 curation strategy 的标准 testbed。[1][2]

实验层	固定什么	改变什么	关键指标	决策
Extractor ablation	模型、token、seed 框架	WET vs WARC+extractor	aggregate benchmark / PPL	选择正文抽取器
Filter ablation	同规模同 token	规则/阈值/分类器	下游分 + retained ratio	看单位 token 收益
Dedup ablation	同源同 token	global/per-snapshot/ threshold	模型分 + unique tokens	选择冗余控制策略
Mixture ablation	总 token 固定	domain weights	任务向量 + domain PPL	选择采样配方
Horizon ablation	模型固定	训练 token 长度	性能曲线斜率	判断数据是否“耗尽”

最小可行实验：不要直接用最终 7B/70B 模型验证数据。先用 100M~2B proxy 模型、固定训练 token、至少 2 个随机 seed 做相对比较，再把最优候选上推到更大 scale。数据实验的目标首先是排序（ranking）而不是复现最终绝对分数。

19. Data Observability：你需要一套“数据版训练监控”

指标类别	建议指标	为什么重要
Volume	documents / bytes / tokens / unique tokens	区分规模与可用信息量
Retention	每阶段保留率、按 source/lang/domain 分解	定位哪个过滤器改变分布
Duplication	cluster size、重复率、重复来源图	识别站群/模板/镜像
Quality	score histogram、bucket 占比、reference fit	监控质量漂移
Language	lang distribution + confidence	避免低资源语言被误杀
Safety/PII	命中率、类别、来源域分布	治理与风险审计
Contamination	每 benchmark overlap rate	保证评测可信
Freshness	crawl/year distribution	控制知识时效与版本

指标类别	建议指标	为什么重要
Provenance	source→document→derived sample lineage completeness	支持删除、复现和问题追踪
Training value	proxy model delta / token	最终判断数据是否值得保留

20. 四条公开管线的设计哲学对比

体系	核心定位	关键策略	最值得复用的经验
Dolma	透明、多源、可复现实验语料	Web+code+books+academic+reference; 多阶段 dedup/filter	保留完整 source mixture; 对每一层处理做文档与 ablation
FineWeb	把 Common Crawl Web 做到高质量大规模	WARC 重抽取; per-snapshot MinHash; C4+自研 heuristics; Edu classifier	不要迷信 global dedup; 用小模型决定过滤器, 而非主观样本检查
DCLM	把数据集设计变成 benchmark/testbed	标准 240T pool; 模型化 filtering; 广泛 curation 对比	quality filter 的“正样本定义”是核心超参; 数据设计可带来接近模型/算力升级的收益
Nemotron-CC	解决 long-horizon 的质量-数量矛盾	classifier ensemble; quality buckets; synthetic rephrase; 更多 unique real tokens	短 horizon 最优过滤器不一定适合 15T token; 绝对高质量 token 数和 long-tail 同样重要

21. 可落地的 Production Blueprint：从 0 到 1 构建预训练数据管线

阶段	动作	产物	验收标准
A. Source Registry	登记来源、许可、时间、语言、采集方式	source_manifest	每个 document 可追溯到 source/version
B. Raw Ingest	原样保存 WARC/PDF/Git/dump	immutable raw layer	可重放、checksum 完整
C. Canonical Parse	正文/结构抽取 + metadata	canonical_document	抽取 QA + boilerplate 率下降
D. Cheap Filters	encoding/lang/URL/明显垃圾	filtered_v0	高 recall; 不做激进质量判断
E. Early Dedup	URL/document exact	filtered_v1	降低后续成本, 保留 provenance cluster
F. Quality Features	rules/PPL/classifier/LLM labels	feature store	所有 score 可追溯版本
G. Fuzzy Dedup	MinHash/substr/semantic 候选	dedup graph	cluster 分布可解释
H. Governance	PII/license/safety/decontam	policy views	不同 policy 可生成不同 dataset view
I. Quality Buckets	Q1~Q5 + domain/lang tags	curated pool	不提前把可用长尾永久删除
J. Mixture Search	proxy ablation / DoReMi / RegMix	mixture recipe	固定 token 下有统计稳定收益
K. Training Export	tokenize/pack/shard	train shards	hash、版本、token 数、sampling config 完整

22. 推荐的数据对象模型与管线接口

```
Document {
  doc_id, source_id, source_version, url_or_path, timestamp,
  raw_pointer, canonical_text, language, language_score,
  domain_tags, quality_features{}, quality_scores{},
  dedup_cluster_ids{}, license, pii_flags{}, safety_flags{},
  contamination_flags{}, lineage[], parser_version, filter_version
}
```

设计重点是“feature first, policy later”：尽可能把语言、质量、重复、风险、许可等信息保存为可版本化 feature，而不是在早期一步就永久删除。这样可以针对不同模型目标构建不同 dataset view，并允许未来重新调整阈值。

23. 十类最常见失败模式

失败模式	表象	根因	修复
只追求规模	token 数很大但模型不涨	重复/垃圾/模板占比高	unique token + proxy value 指标
只追求高质量	短测很好，长训早早饱和	过度过滤导致数据不足	quality buckets + horizon-aware mixture
全局去重过猛	旧数据/小语种突然消失	dedup 改变边际分布	per-source/crawl dedup + retain policy
分类器过拟合风格	模型只会“百科/解释体”	正样本单一	多 reference + ensemble + diversity audit
英语规则平移	其他语言大量误删	统计特征跨语言失效	language-adaptive thresholds
PDF 纯文本化	STEM 数据看似很多但能力弱	公式/表格/顺序被破坏	layout-aware / math-aware extraction
Code 按文件处理	模型缺 repo 上下文、重复高	fork/vendor/generated 未处理	repo-aware pipeline
Synthetic 无验证	训练分数不稳定/事实性变差	生成错误积累	grounding + verifier + human-data anchor
无 provenance	无法删除、解释和重建	数据对象设计缺陷	immutable raw + lineage
只看人工样本	“看着好”但模型不涨	人类质量 ≠ 训练价值	固定模型/token 的 ablation

24. 选型决策表：目标不同，Data Pipeline 应不同

目标模型	数据重点	过滤策略	混合策略	额外防线
通用 Base LLM	Web 多样性 + reference + code	中等强度；保留多 bucket	Web 主导，reference/code 补强	去重、污染、PII、长尾保护
Coding LLM	repo code + docs + issues + general text	代码专用质量/许可/生成文件过滤	code 高权重，保留自然语言解释	repo dedup、secret scan、license
Math/Reasoning	math/science/code + high-quality web	结构保真优先于过滤强度	上采样 math/science，保留 general	公式解析、eval contamination
Multilingual	多语 Web + local reference	语言自适应、低资源高 recall	temperature/质量 × 重复重平衡	LID 审计、文化/语言多样性
Long-horizon 10T+	大量 unique real tokens + quality buckets	避免 top-10% 一刀切	阶段化 curriculum	重复收益曲线、synthetic 比例控制

25. 最值得沉淀的 12 条 Know-Why / Know-How

- 1. 数据管线真正控制的是 P_train(x)，不是“文件是否干净”。
- 2. Extraction 本身就是质量过滤；先把正文抽对，再谈后续评分。
- 3. 过滤阈值不要从论文抄数字，要复制“统计分布→候选规则→小模型 ablation”的方法。
- 4. 质量分类器的正样本定义就是隐式能力目标；它决定模型偏向什么文体和知识结构。
- 5. Dedup 会改变来源和时间分布，因此“保留哪个副本”与“删哪个副本”同样重要。
- 6. 高质量比例与高质量 token 的绝对数量是两种指标；长 horizon 更关心后者与 unique tokens。
- 7. Web 不是低质量来源，它是高方差来源；工程目标是把高方差拆成可控 quality buckets。
- 8. Reference/Books/Academic 的价值在于补齐稳定、编辑密集的分布，而不是完全替代 Web。

9. Math/Code 的瓶颈经常是结构抽取与去重，而不是“有没有足够网页”。
10. Synthetic data 最适合作为 grounded transformation / targeted augmentation，不应视为无限真实数据替代品。
11. Multilingual pipeline 必须语言自适应；英语 heuristic 和 classifier 不能无损外推。
12. Data ablation 是数据团队的核心研发闭环：固定模型、token、训练设置，只改变数据，并用真实能力指标排序。

参考资料与证据来源

- [1] Penedo et al. — The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. arXiv:2406.17557, 2024.
- [2] Li et al. — DataComp-LM: In search of the next generation of training sets for language models. arXiv:2406.11794, 2024.
- [3] Soldaini et al. — Dolma: an Open Corpus of Three Trillion Tokens for Language Model Pretraining Research. arXiv:2402.00159, 2024.
- [4] Penedo et al. — The RefinedWeb Dataset for Falcon LLM. arXiv:2306.01116, 2023.
- [5] Lee et al. — Deduplicating Training Data Makes Language Models Better. arXiv:2107.06499, 2021/2022.
- [6] Xie et al. — DoReMi: Optimizing Data Mixtures Speeds Up Language Model Pretraining. arXiv:2305.10429, 2023.
- [7] Liu et al. — RegMix: Data Mixture as Regression for Language Model Pre-training. arXiv:2407.01492, 2024.
- [8] Penedo et al. — FineWeb2: One Pipeline to Scale Them All - Adapting Pre-Training Data Processing to Every Language. arXiv:2506.20920, 2025.
- [9] Nguyen et al. — CulturaX: A Cleaned, Enormous, and Multilingual Dataset for LLMs in 167 Languages. arXiv:2309.09400, 2023.
- [10] Gao et al. — The Pile: An 800GB Dataset of Diverse Text for Language Modeling. arXiv:2101.00027, 2020.
- [11] Kocetkov et al. — The Stack: 3 TB of permissively licensed source code. arXiv:2211.15533, 2022; and Li et al., StarCoder, arXiv:2305.06161.
- [12] Paster et al. — OpenWebMath: An Open Dataset of High-Quality Mathematical Web Text. arXiv:2310.06786, 2023.
- [13] Su et al. — Nemotron-CC: Transforming Common Crawl into a Refined Long-Horizon Pretraining Dataset. arXiv:2412.02595, v2 2025.
- [14] Karimi Mahabadi et al. — Nemotron-CC-Math: A 133 Billion-Token-Scale High Quality Math Pretraining Dataset. arXiv:2508.15096, 2025.
- [15] Gunasekar et al. — Textbooks Are All You Need. arXiv:2306.11644, 2023.
- [16] Shumailov et al. — AI models collapse when trained on recursively generated data. Nature 631, 755-759, 2024.
- [17] Raffel et al. — Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (C4/T5). JMLR 21, 2020.
- [18] Wenzek et al. — CCNet: Extracting High Quality Monolingual Datasets from Web Crawl Data. arXiv:1911.00359, 2019/2020.
- [19] Dodge et al. — Documenting Large Webtext Corpora: A Case Study on C4. EMNLP 2021.
- [20] Laurençon et al. — The BigScience ROOTS Corpus: A 1.6TB Composite Multilingual Dataset. arXiv:2303.03915, 2023.
- [21] Team OLMo — 2 OLMo 2 Furious. arXiv:2501.00656, 2025.
- [22] Meta AI — The Llama 3 Herd of Models. arXiv:2407.21783, 2024.
- [23] Qwen Team — Qwen2.5 Technical Report. arXiv:2412.15115, 2024.
- [24] Weber et al. — RedPajama: an Open Dataset for Training Large Language Models. arXiv:2411.12372, 2024.
- [25] Jiang et al. — Investigating Data Contamination for Pre-training Language Models. arXiv:2401.06059, 2024.

阅读顺序建议：若只读 6 篇：FineWeb [1] → DCLM [2] → Dolma [3] → Dedup [5] → DoReMi/RegMix [6][7] → Nemotron-CC [13]。这组资料基本覆盖 extraction、filtering、dedup、mixture 与 long-horizon 的核心矛盾。